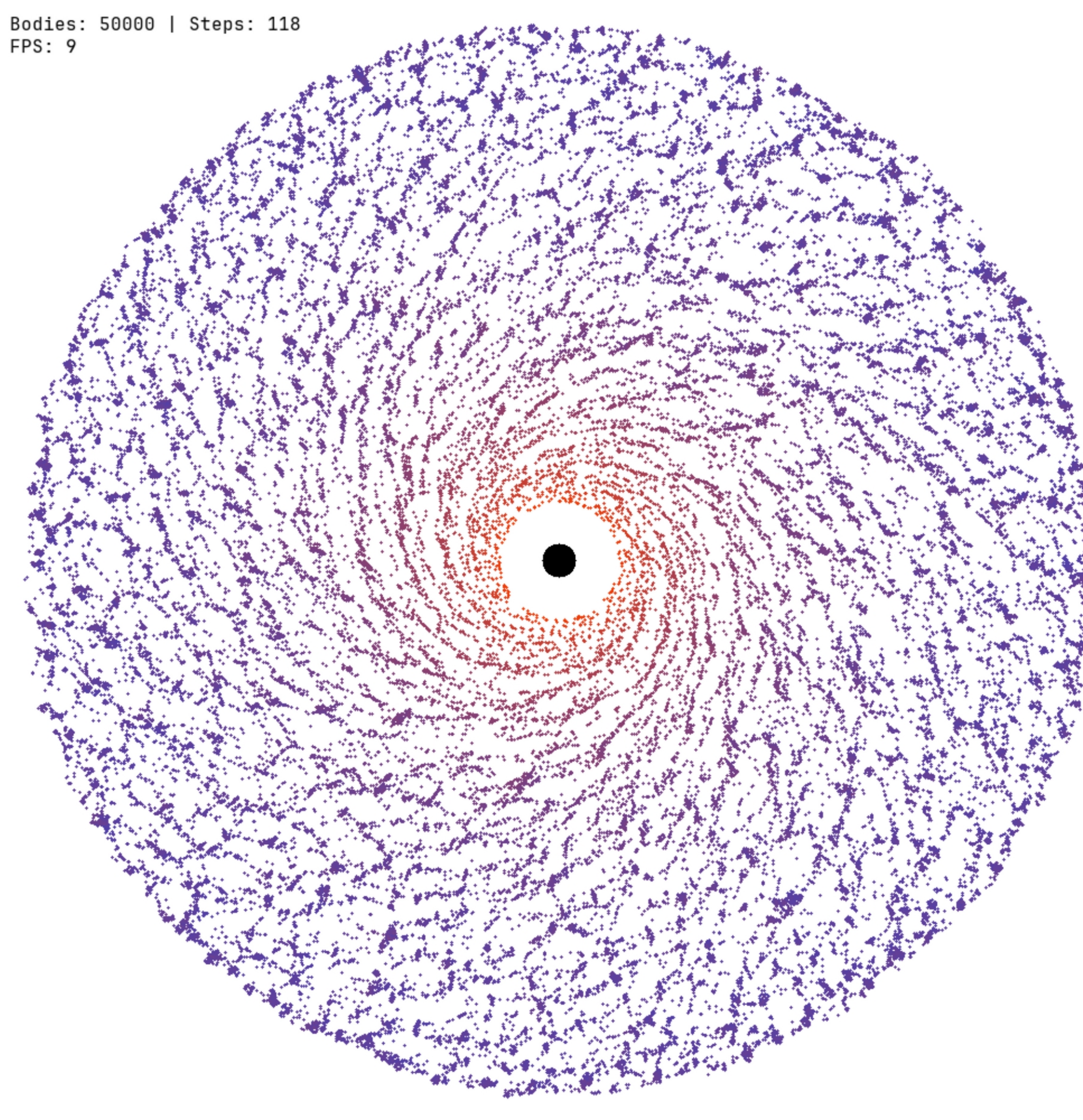
Numerical Performance Analysis of the Direct and Barnes-Hut Algorithm for Solving the N-Body Problem

By:
Balarezo J. Gabriel

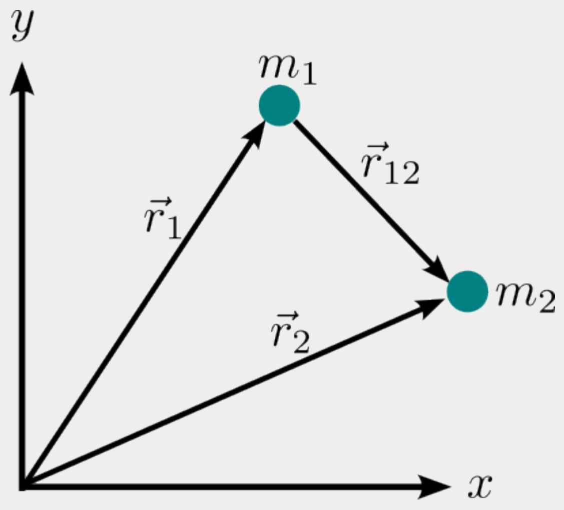
Advaced Programming

June 3, 2025



Agenda

- The Barnes-Hut Approximation
- Results
- Conclusions
- References



$$\mathbf{F}_{ij} = - \sum_{j=1, j \neq i}^{N} \frac{G m_i m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

The Barnes-Hut approximation

The Barnes-Hut algorithm is an approximation algorithm for performing an N-body simulation with a time complexity of $\mathrm{O}(\mathrm{N}\log N)$

Direct method

$$\frac{d^2\mathbf{r}_i}{dt^2} = -\sum_{j=1, j\neq i}^{N} \frac{Gm_j}{|\mathbf{r}_{ij}|^3}\mathbf{r}_{ij}$$

BH-approximation

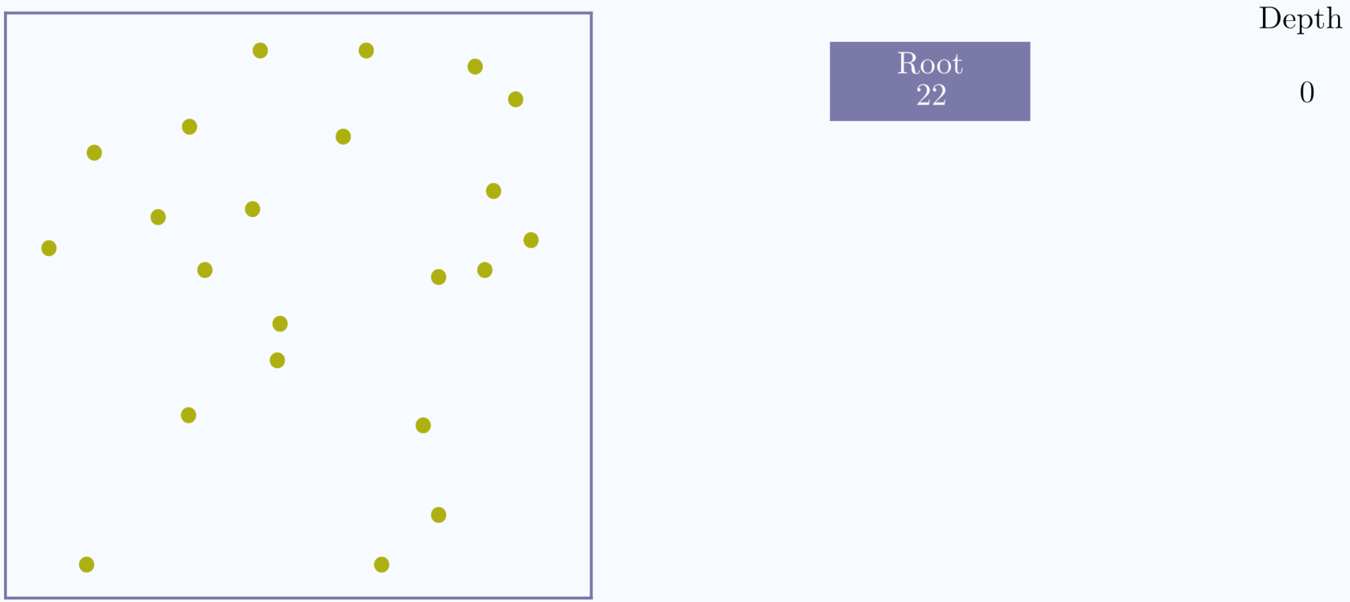
$$M_{cm} = \sum_{i=1}^{n} m_i \qquad \mathbf{r}_{cm} = \frac{1}{M}\sum_{i=1}^{n} m_i\mathbf{r}_i$$

$$\frac{d^2\mathbf{r}_i}{dt^2} = -\frac{GM_{cm}}{|\mathbf{r}_{ij}|^3}\mathbf{r}_{ij}$$

The Barnes-Hut approximation

How does it work?...

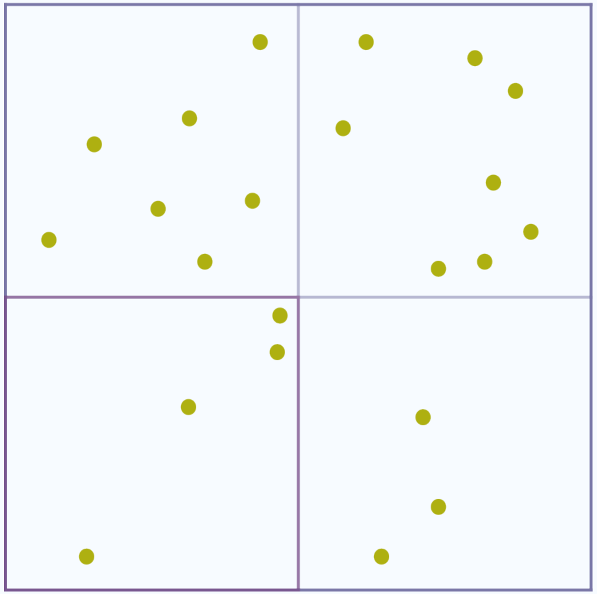
It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions. The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body.

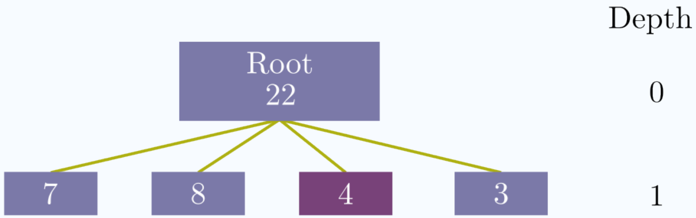


The Barnes-Hut approximation

How does it work?...

It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions. The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body.

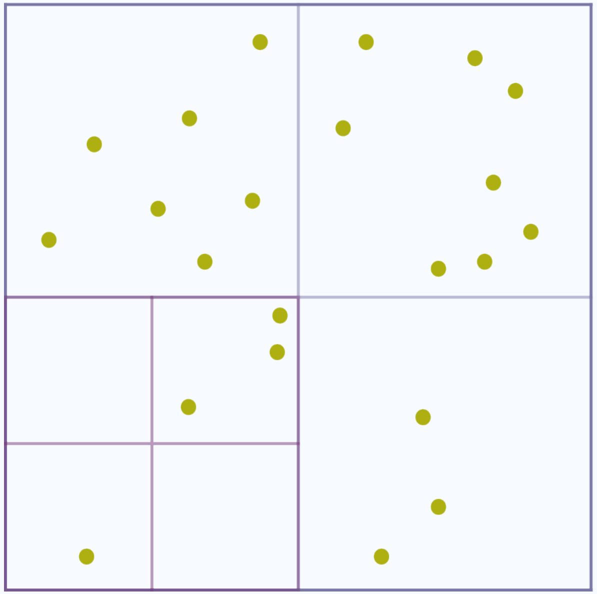


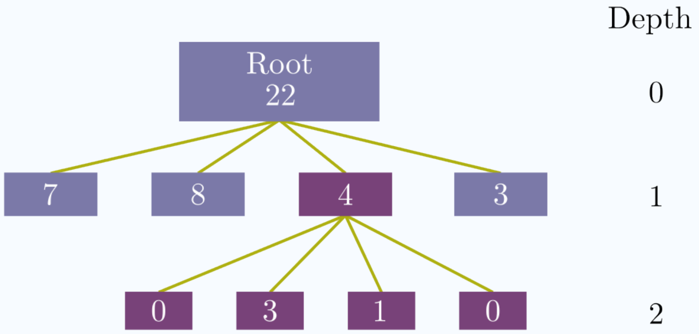


The Barnes-Hut approximation

How does it work?...

It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions. The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body.

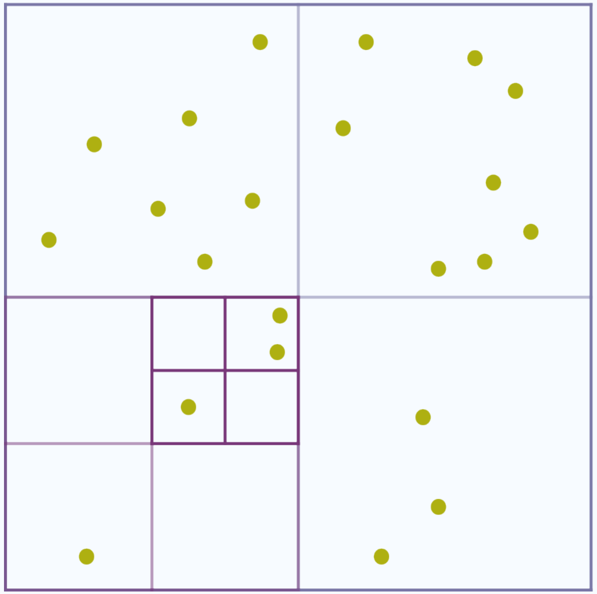


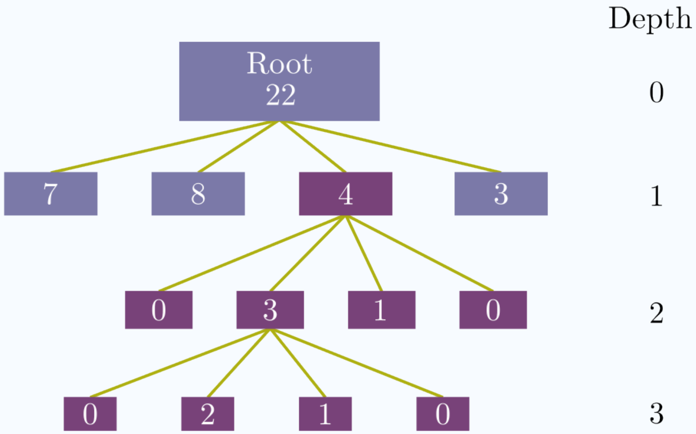


The Barnes-Hut approximation

How does it work?...

It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions. The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body.

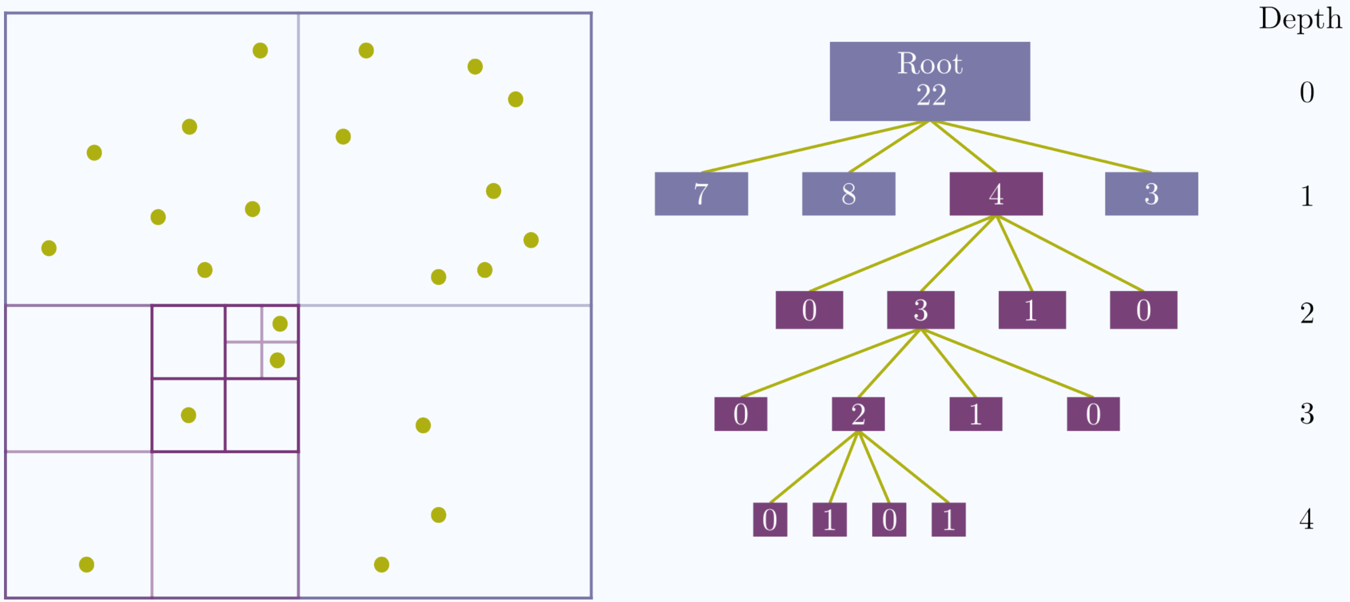




The Barnes-Hut approximation

How does it work?...

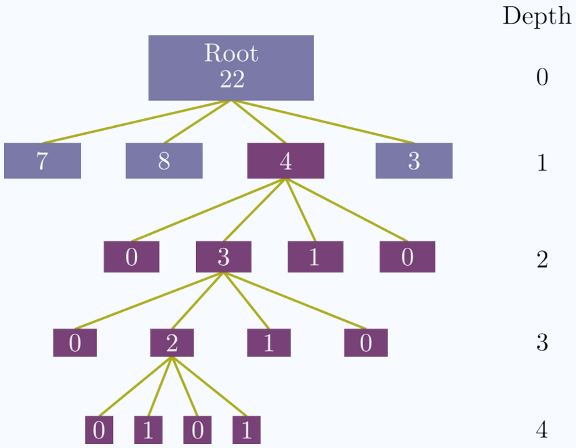
It utilises a hierarchical tree structure—a quadtree in 2D and an octree in 3D—to represent the spatial distribution of bodies and their interactions. The tree is built by recursively subdividing the simulation area into quadrants until each node contains either zero or one body.

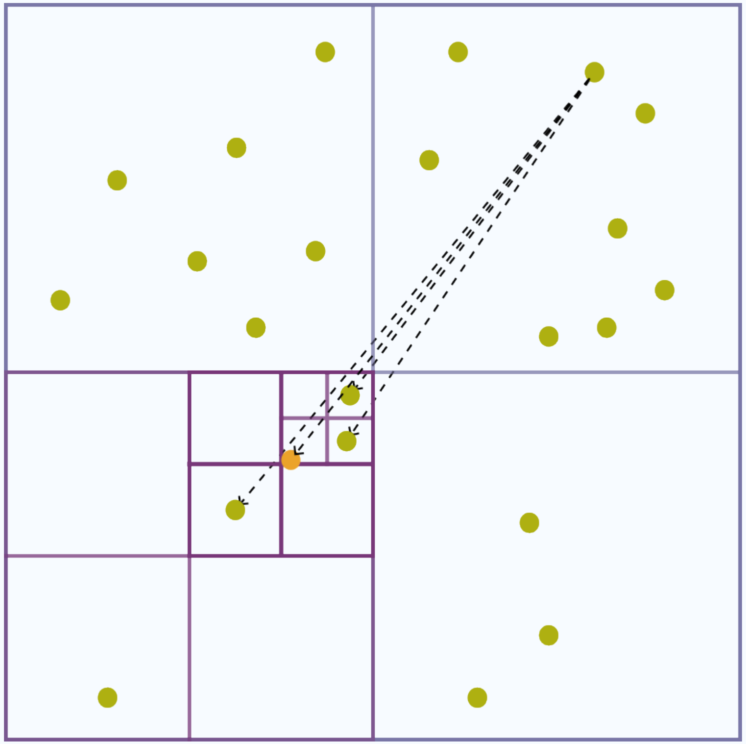


The Barnes-Hut approximation

- Each node stores its centre of mass and the total mass of the bodies contained in the node.
- We compute the distance d between m_i and the centre of mass of the node. We also keep track of the size s of the node, i.e. the length of one side of the quadrant represented by the node.
- Use the θ criterion to decide if a node is sufficiently far away or if we need to traverse its child nodes.

$$\frac{s}{d} < \theta$$





The Barnes-Hut approximation

How do we update our system?

Verlet Integrator

Verlet integration is a numerically stable, time-reversible, and conservative method , making it well-suited for simulations of this nature.

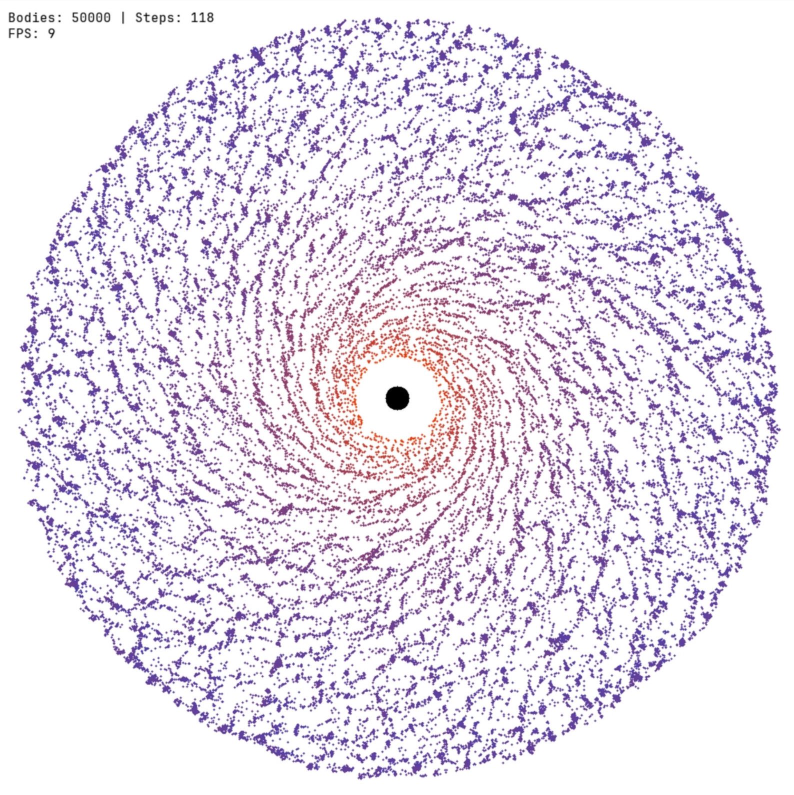
$$\mathbf{r}_i(t + \Delta t) = 2\mathbf{r}_i(t) - \mathbf{r}_i(t - \Delta t) + \mathbf{a}_i(t)\Delta t^2 + \mathcal{O}(\Delta t^4)$$

Since Verlet is not a self-starting method, we utilise a suitable approximation to perform the first update.

$$\vec{r}_i(t_1) = \vec{r}_i(t_0) + \vec{v}_i(t_0)\Delta t + \frac{1}{2}\vec{a}_i(t_0)\Delta t^2 + \mathcal{O}(\Delta t^3)$$

Results

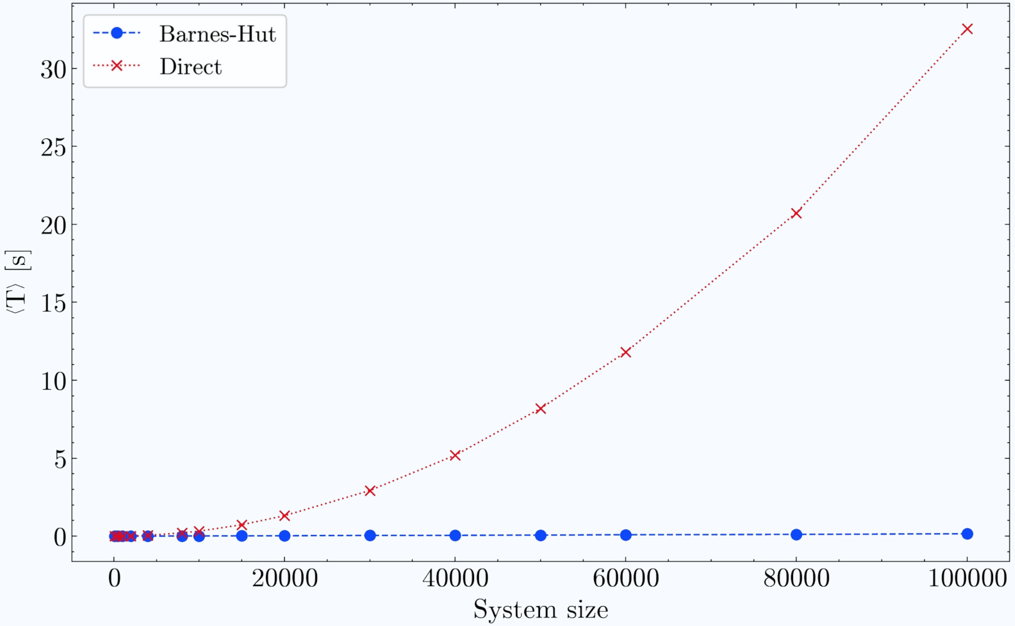
Each system size was simulated using both methods for 100 time steps.
Execution times were recorded for each time step and averaged afterwards.



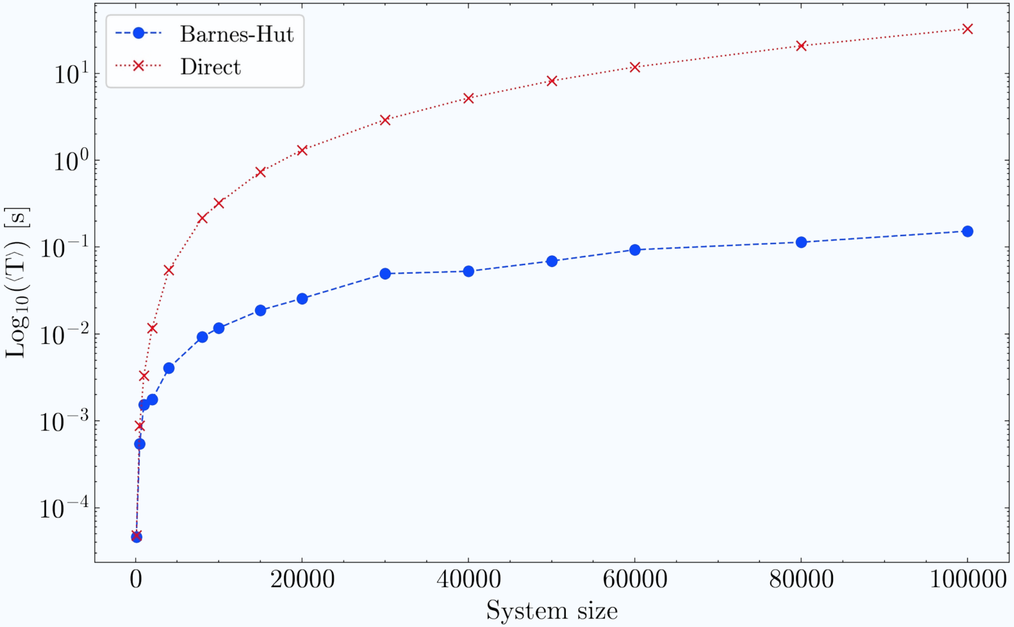
Benchmark results

Number of Particles	Direct (s)	Barnes-Hut (s)	Speedup
100	0.000048	0.000046	1.04
500	0.000877	0.000542	1.62
1000	0.003328	0.001538	2.16
2000	0.011730	0.001769	6.63
4000	0.054317	0.004049	13.41
8000	0.215674	0.009213	23.41
10000	0.320299	0.011732	27.29
15000	0.734168	0.018731	39.19
20000	1.310772	0.025587	51.23
30000	2.925272	0.049420	59.18
40000	5.189255	0.052630	98.61
50000	8.185188	0.069083	118.48
60000	11.801987	0.093106	126.75
80000	20.728767	0.113791	182.19
100000	32.540323	0.152524	213.36

Results

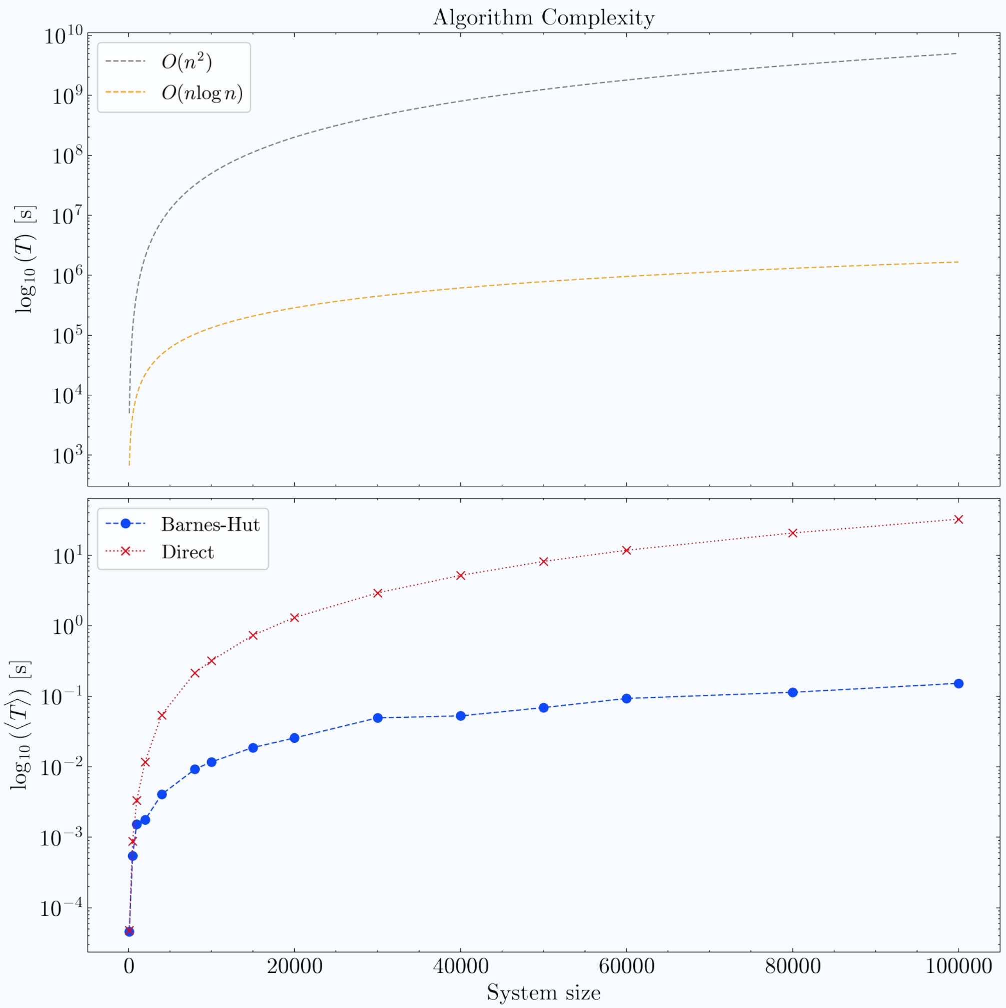


Barnes-Hut
Direct
30
25
20
15
10
5
0
⟨T⟩ [s]
0
20000
40000
60000
80000
100000
System size



Barnes-Hut
Direct
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
$\mathrm{Log}_{10}(\langle T \rangle)$ [s]
0
20000
40000
60000
80000
100000
System size

Results



Algorithm Complexity
$O(n^2)$
$O(n\log n)$
$\log_{10}(T)$ [s]
10^{10}
10^9
10^8
10^7
10^6
10^5
10^4
10^3
Barnes-Hut
Direct
$\log_{10}(\langle T\rangle)$ [s]
10^1
10^0
10^{-1}
10^{-2}
10^{-3}
10^{-4}
0
20000
40000
60000
80000
100000
System size

Conclusions

- The direct method provides high accuracy but scales poorly with system size due to its $\mathrm{O}(\mathrm{N}^2)$ complexity, making it suitable only for small-scale simulations.
- The Barnes-Hut algorithm dramatically reduces computational cost for large systems, achieving over 200× speedup at N=10^4, while maintaining reasonable accuracy.
- By using a hierarchical tree structure and the theta criterion, Barnes-Hut achieves $\mathrm{O}(\mathrm{N}\log N)$ scaling, making it ideal for astrophysical and cosmological N-body simulations.
- Leveraging OpenMP for parallel force computation and integration further enhances both methods' performance—especially critical for real-time or large-scale scenarios.

Thanks for your attention!!

Any questions?